

CCS2300 – Data Structures & Algorithms

Railway Reservation System

Group 14



Rashith Randil – CIT-25-01-0177
K.H.Kaveesha – CIT-25-01-0755
Sakunika Kavindi – CIT-25-01-0769
Tharushi Dewmini – CIT-25-01-0828
Oshini Akarsha – CIT-25-01-0084
Dinaya Ranasinghe – CIT-25-01-0648

Submission Date:

16/08/2026

1. Introduction

1.1 Project Background

The Railway Reservation System based on Java and it designed to manage railway related information like trains, stations, users, bookings and seat availability.

Our project demonstrates the practical use of data structures and algorithms in real-world railway reservation system. We used a few different data structures to help store and sort the information effectively.

The whole system was built by using Java and it follows an object oriented approach. The primary model classes inside the project are Train, Booking, Station, User and Ticket

1.2 Purpose of the Project

The main purpose of the system is to give an organized method to manage railway information and to show how data structures can be applied to a railway reservation system.

The system is planned to support operations such as :

- Managing railway stations
- Managing train information
- Searching for trains
- Searching for stations
- Adding and deleting trains
- Managing bookings
- Managing seat availability
- Searching bookings
- Demonstrating tree traversals

1.3 Project Objectives

The main objectives of the project are :

1. To develop a railway management system using Java.
2. To apply various data structures to a practical problem.
3. To implement searching and tree operations.
4. To demonstrate the use of arrays and linked lists.
5. To implement a Binary Search Tree for train management.
6. To implement an AVL Tree for booking management.
7. To provide validation for invalid input.
8. To prevent duplicate train and station records.
9. To manage train seat availability.
10. To analyze the time and space complexity of the implemented structures and algorithms.

2. Problem Description

2.1 Problem being solved

Railway reservation involves handling a lot of information like trains, stations, users, bookings, seat availability and routes. As the number of users and train records grows, managing these records becomes hard.

The main problem this project is trying to solve is creating a Railway Reservation System that can handle train and reservation activities in a way. The system offers features like adding and managing trains looking for trains making and cancelling bookings keeping track of seats dealing with passengers waiting for seats and finding routes between stations.

The project also wants to show how data structures and algorithms can be used in life to solve these problems. The task requires using data structures and algorithms written in Java.

2.2 Existing problems

Old or simple railway reservation methods can cause problems when dealing with a lot of records. Some of the issues are:

- **Hard to manage records:** Keeping track of train, user and booking records by hand is not easy.
- **Slow search:** Looking for records one by one takes time as the number of records goes up.
- **Seat management problems:** Without tracking the seat might be given to two different people.
- **Booking and cancellation issues:** Keeping track of reservations and cancellations by hand can lead to mistakes.
- **Waiting list management:** People who can't get seats need to be kept in an order.
- **Route management:** Stations and their connections form a network that needs a way to find routes.
- **Sorting issues:** Train details might need to be arranged based on things, like train ID, train name or available seats.
- **Duplicate records:** It's important to stop having users, seats or other records.

So the right data structures and algorithms are needed to make the reservation system work better and be more reliable. The project specifically asks for structures like linked lists, stacks, queues, hash tables, sets, trees and graphs.

2.3 Proposed solution

- The solution is a Railway Reservation System built in Java that uses the data structures and algorithms to manage railway activities
- A linked list is used to handle train records that can change over time while arrays are used for information that doesn't change size. A stack is used for booking processes. A queue is used to manage people waiting for seats. A set helps stop the seat from being given a hash table makes it easier to store and find user information.
- A Binary Search Tree and an AVL Tree are included to help with searching and organizing data. A graph is used to show the network of railway stations allowing route operations to be done using BFS and DFS algorithms.
- The system also uses Linear Search, Bubble Sort, Selection Sort, Insertion Sort, Merge Sort and Quick Sort to keep information organized and easy to find. These algorithms can be compared based on how they work and how memory they use and which ones work best for different tasks.
- Overall the system offers a way to handle railway reservations. Shows how data structures and algorithms can be used to make the system better in terms of efficiency organization and reliability.

3. System Design

3.1 System Architecture

The project is organized into three main packages:

model

Contains the main entities of the system:

- **Train.java**
- **Station.java**
- **Booking.java**
- **User.java**
- **Ticket.java**

structure

Contains the data structure implementations:

- **StationArray.java**
- **TrainLinkedList.java**
- **TrainBST.java**
- **BookingAVLTree.java**
- **BookedSeatsSet.java**
- **BookingStack.java**
- **StationRouteGraph.java**
- **UserHashTable.java**

app

Contains application/menu classes:

- **Main.java**
- **AdminMenu.java**
- **UserMenu.java**

The application package is intended to provide the user interface and connect the model classes with the data structures.

3.2 Main modules

Train Management

The Train class stores:

- **Train ID**
- **Train name**
- **Starting station**
- **Total seats**
- **Available seats**

The system supports reserving and releasing seats.

Station Management

The station class stores:

- **Station ID**
- **Station name**
- **Station address**

StationArray stores station objects in an array.

Booking Management

The booking class stores

- **Booking ID**
- **User ID**
- **Train ID**
- **Starting location**
- **Destination**
- **Number of seats**
- **Seat number**
- **Booking status**

The booking status can be:

- **CONFIRMED**
- **WAITLISTED**
- **CANCELLED**

User Management

The User class store:

- **Username**
- **Full name**
- **Email**
- **Password**
- **Phone**
- **Booking list**

3.3 System Workflow

A simplified workflow is:

1. The user can first enters or accesses railway information.
2. Train and station information is stored in appropriate data structures.
3. The user can searches train or route.
4. Once user select train, a booking is created.
5. System checks whether seat availability.
6. Seats are reserved if available.
7. Booking information is stored in system.
8. Users can search for booking details using the AVL Tree.
9. Train information can be searched, added, or deleted using the Binary Search Tree (BTS).
10. Station information can be searched using linear search.

4. Data Structures Used

4.1 Array

- An array is used to store a fixed collection of railway data where elements need to be accessed using an index.
- In our Railway reservation system, arrays are used in the ***StationArray*** to store and manage station information. Each station can be accessed directly using its index.

Why we use it:

- Provides fast access to station data using an index.
- Simple and efficient for storing a fixed or limited number of stations.
- Useful when sequential station information needs to be maintained.

Example:

StationArray stores station objects such as station ID, station name, and related information.

4.2 Linked List

- A Linked List is used to store train information dynamically. Each node contains a train object and a reference to the next node.
- In our Railway reservation system, the ***TrainLinkedList*** is used to maintain a collection of trains and support operations such as adding and searching for trains.

Why we use it?

- Allows dynamic addition of train records
- Make insertion and deletion easier compared with a fixed-size array
- Doesn't require a fixed size
- Useful for maintaining a changing list of trains

Example:

When a new train is added to the system, it can be inserted into the ***TrainLinkedList***.

4.3 Stack

- A stack follows the LIFO principle. The most recently add item is removed first. Our system implements the stack using linked nodes.

Push operation

- When a new booking action occurs, it is added using:
recentAction.push(booking, "BOOKED");
- The new booking becomes the new top of the stack.

Pop operation

The pop() operation removes the most recent booking.

Booking booking = top.booking;

Top = top.next;

Peek operation

The peek() method views the latest booking without removing it.

The Stack is useful for:

- Tracking recent booking activities
- Maintaining the latest actions
- Displaying recent activities
- Demonstrating LIFO behavior

4.4 Queue

- A Queue follows the FIFO principle.
- The passenger who joins the waiting list first should receive the available seat first.

Enqueue

- When a train has no available seats, a booking is added to the waiting list:
waitingList.enqueue(booking);
- The booking is inserted at the rear.

Dequeue

- When a seat becomes available, the first passenger is removed:
Booking candidate = waitingList.dequeue();
- The booking at the front is removed first.

4.5 Hash Table

The UserHashTable stores user information and uses the username as the key.

How works hashtable

- When a user is added, the username is passed through a hash function. The system is using a polynomial-style hash:
 $hash = (hash * 31 + username.charAt(i)) \% capacity;$

Why Hash Table is used

The Hash Table is suitable for user management because it provides:

- Fast average-case user searching
- Fast user insertion
- Fast user removal
- Username-based lookup
- Efficient login processing

4.6 Set ADT

- This system uses the BookedSeatSet class to ensure that the same seat cannot be booked twice on the a booked seat is represented using a combined key:

Train ID + Seat Number

Main purpose

- The most important purpose of this set ADT is duplicate prevention.
Train = T001
Seat = S05 is already booked.
- If another passenger tries to book:
T001 + S05 the system checks:
bookedSeats.contains(trainId, seat);
- If the seat already exists, the booking is rejected

4.7 Binary Search Tree

- We use TrainBST to this system to organize trains according to their Train IDs.

BST Rule

The Binary Search Tree follows this rule.

- Smaller Train ID → Left
- Larger Train ID → Right

Searching

- The system can search for a train using:
trainIndex.search(trainId);
- The search compares the requested Train ID with the current node.
- If it is smaller, it moves left.
- If it is larger, it moves right

Insertion

- When a train is added to the system:
trainIndex.insert(train);
- The train is positioned according to its Train ID.

Traversals

The implementation supports:

- In-order
- Pre-order
- Post-order

In-order traversal is especially useful because it displays the Train IDs in sorted order.

Why BST is used

The BST provides an organized tree-based index for train records.

It is useful for:

- Searching trains
- Inserting trains
- Deleting trains
- Sorting through in-order traversal
- Demonstrating tree-based data structures

4.8AVL Tree

In our system, the AVL Tree is not TrainAVL.

BookingAVLTree -: Stores Booking objects and uses the Booking ID as the key.

What is an AVL Tree?

- An AVL Tree is a self-balancing Binary Search Tree. After every insertion or deletion, the tree checks its balance factor.

Searching

- The system searches for a booking using.
bookings.search(bookingId);
- This is used when a user wants to find a specific booking.

Insertion

- When a booking is successfully created.
Booking.insert(booking);
- The booking is inserted according to its Booking ID.

Deletion

- When a booking is cancelled:
Booking.delete(booking.getBookingID());
- The booking is removed from the AVL tree.

Why AVL Tree is used

- The AVL Tree is useful because it maintains a balanced structure even when many bookings are added.

It provides efficient:

- Booking insertion
- Booking searching
- Booking deletion

The main advantage over a normal BST is that the AVL tree prevents the tree from becoming highly unbalanced.

4.9Graph

The **StationRouteGraph** represents the railway network. Our system maintains station information and connections using maps.

Map<String, Station> stations

Map<String, Map<String, Double>> graph

The graph stores the distance between connected stations.

Example:

CMB → GPH = 20 km

GPH → VGD = 25 km

Adding routes

The system creates a two-way connection. A route can be added using

Routes.addRoute(firstStationId, secondStationId, distance);

Distance calculation

The Graph stores distances between stations.

Example-:

Routes.calculateDistance(route);

Why Graph is used

- A railway network is naturally represented as a graph because stations have relationships with other stations.

The Graph supports:

- Station connections
- Route management
- Route searching
- Connectivity checking
- Distance calculation
- BFS
- DFS

5. Algorithms Used

5.1 Linear Search

Linear search is implemented in *StationArray.java*.

The algorithm checks each stop one by one until the needed station is eventually found

Example:

```
for (int i = 0; i < size; i++) {  
    if (stations[i].getStationName().equalsIgnoreCase(stationName)) {  
        return stations[i];  
    }  
}
```

Linear search is also implemented in *TrainLinkedList.java* for train ID searching.

Complexity

- Base case : $O(1)$
- Average case : $O(n)$
- Worst case : $O(n)$
- Space complexity : $O(1)$

5.2 Bubble Sort

Bubble sort is used in *TrainLinkedList.java*. This is used to line up trains based on their Train ID and compare nearby trains and if they are in the wrong order then they get swapped .

Complexity:

- Best case : $O(n)$
- Average case : $O(n^2)$
- Worst case : $O(n^2)$
- Space complexity : $O(1)$

5.3 Selection Sort

Selection sort is used in *TrainLinkedList*. It is used to arrange the trains by their Train Name, in alphabetical order, finds the smallest name, and then places it, in the right position.

Complexity :

- Best case : $O(n^2)$
- Average case : $O(n^2)$
- Worst case : $O(n^2)$
- Space complexity : $O(1)$

5.4 Insertion Sort

Insertion sort is used in **TrainLinkedList**. It is used to arrange trains according to their available seats. It is useful when the data is already partly sorted.

Complexity:

- Best case : $O(n)$
- Average case : $O(n^2)$
- Worst case : $O(n^2)$
- Space complexity : $O(1)$

5.5 Merge Sort

Merge sort is used in **TrainLinkedList**. It is used to sort trains according to their total number of seats. It is useful because it provides good performance even when there are many trains.

Complexity:

- Best case : $O(n \log n)$
- Average case : $O(n \log n)$
- Worst case : $O(n \log n)$
- Space complexity : $O(n)$

5.6 Quick Sort

Quick sort is used in **TrainLinkedList**. It is used to arrange trains according to their starting station. It is generally fast for sorting large amounts of data.

Complexity :

- Best case : $O(n \log n)$
- Average case : $O(n \log n)$
- Worst case : $O(n^2)$
- Space complexity : $O(\log n)$

5.7 BST Operations

The BST uses **Insert** to add trains, **Search** to find trains and **Delete** to remove trains. **Inorder** displays trains in sorted order, while **Preorder** and **Postorder** displays the tree in various orders.

5.8 AVL Tree Operations

In AVL tree **Insert** helps to add bookings, **Search** helps to find booking and **Delete** helps to remove bookings. Rotation are used to keep the tree balanced while **Inorder**, **Preorder**, **Postorder** traversals are used to display the bookings in different orders.

Rotations

The AVL implementation contains the four standard balancing cases.

1. LL – Left Left
2. RR – Right Right
3. LR – Left Right
4. RL – Right Left

5.9 BFS

BFS is implemented in ***StationRouteGraph***. It is used to find a route between railway with the fewest number of stops. BFS checks nearby station first before moving further.

Complexity:

- Best case : $O(V + E)$
- Average case : $O(V + E)$
- Worst case : $O(V + E)$
- Space complexity : $O(V)$

5.10 DFS

DFS is implemented in ***StationRouteGraph***. It is used to check whether one railway station is connected to another station. DFS follows one route deeply before checking another route.

Complexity :

- Best case : $O(V + E)$
- Average case : $O(V + E)$
- Worst case : $O(V + E)$
- Space complexity : $O(V)$

6. Complexity Analysis

Data structure / Algorithm	Best case	Average case	Worst case	Space complexity
Array Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Array Insert	$O(1)$	$O(1)$	$O(n)$	$O(n)$
LinkedList Search	$O(1)$	$O(n)$	$O(n)$	$O(n)$
LinkedList Delete	$O(1)$	$O(n)$	$O(n)$	$O(n)$
BST Search	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$
BST Insert	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$
BST Delete	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$
AVL Insert	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$
AVL Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$
AVL Delete	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
BFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$
DFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$

7. Testing and Validation

Testing was done to check whether the main functions of the Railway Reservation System were working correctly. We tested the important parts of the system using both valid and invalid inputs.

The main classes tested were [StationArray](#), [TrainLinkedList](#), [BookingStack](#), [BookingWaitingListQueue](#), [UserHahTable](#), [BookedSeatSet](#), [TrainBST](#), [BookingAVLTree](#) and [StationRouteGraph](#).

7.1 Test Cases

Test ID	Class	Test Scenario	Expected Results	Actual Results
TC01	UserHashTable	Register a new user	New User should be registered	User registered successfully
TC02	UserHashTable	Register using the same username	Duplicate username should not be allowed	Duplicate username was rejected
TC03	UserHashTable	User login with correct details	User should be able to log in	Login successfully
TC04	UserHashTable	User login with a wrong password	User login should not be allowed	Login was rejected
TC05	StationArray	Add a new station	New station should be added	Station was added
TC06	StationArray	Add the same station again	Duplicate station should not be added	Duplicate station was rejected
TC07	TrainLinkedList	Search for an existing train	Train should be found	Train was found
TC08	BookedSeatSet	Reserve an available seat	Seat should be reserved	Seat was reserved
TC09	BookedSeatSet	Try to reserve the same seat again	Same seat should not be booked twice	Duplicate seat booking was prevented
TC10	BookingWaitingListQueue	Add a booking when a seat is unavailable	Booking should enter the waiting list	Booking was added to the waiting list
TC11	BookingStack	Store a recent booking activity	Recent activity should be stored	Activity was stored
TC12	TrainBST	Search for a train	Correct train should be found	Train was found
TC13	BookingAVLTree	Search for a booking	Correct booking should be found	Booking was found
TC14	StationRouteGraph	Find the route between connected stations	Route should be found	Route is found

7.2 Validation

This validation step is done in order to make sure that the system does not accept any wrong or duplicate information.

The **UserHashTable** validates whether the username is present in the system before adding another one. Similarly, the **StationArray** will validate whether there are any duplicate station entries before adding a station entry.

The **BookedSeatSet** prevents duplicate seating from being booked in the system. In case the seat is not available, the booking can be put in the **BookingWaitingListQueue**.

The system also validates login credentials before letting the user proceed. In case the username or password is wrong, the login process is declined.

The **StationRouteGraph** is used to check whether there is any connectivity between railway stations before finding the route.

7.3 Testing Summary

The test results revealed that all the classes of the Railway Reservation System were functioning as required. The valid data was successfully processed whereas the duplicate data or invalid data was not accepted by the system.

The testing process has also provided us an opportunity to ensure the functioning of different classes and data structures.

The screenshots and output result are given below in the respective section.

8. Screenshots and Results

Station Management

--- STATION AND ROUTE MANAGEMENT ---

1. View stations
2. Add station
3. Remove station
4. View route network
5. Add route connection
6. Remove route connection
7. Check route using BFS
8. Check connectivity using DFS
0. Back

Choose an option: 1

ID	Station	Address
CMB	Colombo Fort	Colombo
GPH	Gampaha	Gampaha
VGD	Veyangoda	Gampaha
PLG	Polgahawela	Kurunegala
RBG	Rambukkana	Kegalle
KDY	Kandy	Kandy
NWP	Nawalapitiya	Kandy
HTN	Hatton	Nuwara Eliya
NUO	Nanu Oya	Nuwara Eliya
ELL	Ella	Badulla
BDL	Badulla	Badulla

===== SORT TRAINS =====

1. Sort by Train ID (Bubble sort)
2. Sort by Train name (Selection Sort)
3. Sort by Available Seats (Insertion Sort)
4. Sort by Total Seats (Merge Sort)
5. Sort by Start Station (Quick Sort)
0. Back

Choose sorting method: 1

Trains sorted using Bubble Sort.

ID	Train	Route	Available/Total Seats
T001	Podi Menike	Colombo Fort -> Badulla	450/500
T002	Kandy Intercity	Colombo Fort -> Kandy	350/400
T003	Main Line Express	Colombo Fort -> Polgahawela	400/450
T004	Gampaha-Kandy Express	Gampaha -> Kandy	0/300
T005	Udarata Menike	Colombo Fort -> Badulla	300/420
T006	Ella Odyssey	Colombo Fort -> Badulla	200/280
T007	Hill Country Express	Kandy -> Badulla	180/250
T008	Badulla Night Mail	Badulla -> Colombo Fort	250/400
T009	Kandy-Colombo Intercity	Kandy -> Colombo Fort	220/300

===== SORT TRAINS =====

1. Sort by Train ID (Bubble sort)
2. Sort by Train name (Selection Sort)
3. Sort by Available Seats (Insertion Sort)
4. Sort by Total Seats (Merge Sort)
5. Sort by Start Station (Quick Sort)
0. Back

Choose sorting method: 2

Trains sorted using Selection Sort.

ID | Train | Route | Available/Total Seats

T008	Badulla Night Mail	Badulla -> Colombo Fort	250/400
T006	Ella Odyssey	Colombo Fort -> Badulla	200/280
T004	Gampaha-Kandy Express	Gampaha -> Kandy	0/300
T007	Hill Country Express	Kandy -> Badulla	180/250
T002	Kandy Intercity	Colombo Fort -> Kandy	350/400
T009	Kandy-Colombo Intercity	Kandy -> Colombo Fort	220/300
T003	Main Line Express	Colombo Fort -> Polgahawela	400/450
T001	Podi Menike	Colombo Fort -> Badulla	450/500
T005	Udarata Menike	Colombo Fort -> Badulla	300/420

===== SORT TRAINS =====

1. Sort by Train ID (Bubble sort)
2. Sort by Train name (Selection Sort)
3. Sort by Available Seats (Insertion Sort)
4. Sort by Total Seats (Merge Sort)
5. Sort by Start Station (Quick Sort)
0. Back

Choose sorting method: 3

Trains sorted using Insertion Sort.

ID | Train | Route | Available/Total Seats

T004	Gampaha-Kandy Express	Gampaha -> Kandy	0/300
T007	Hill Country Express	Kandy -> Badulla	180/250
T006	Ella Odyssey	Colombo Fort -> Badulla	200/280
T009	Kandy-Colombo Intercity	Kandy -> Colombo Fort	220/300
T008	Badulla Night Mail	Badulla -> Colombo Fort	250/400
T005	Udarata Menike	Colombo Fort -> Badulla	300/420
T002	Kandy Intercity	Colombo Fort -> Kandy	350/400
T003	Main Line Express	Colombo Fort -> Polgahawela	400/450
T001	Podi Menike	Colombo Fort -> Badulla	450/500

===== SORT TRAINS =====

1. Sort by Train ID (Bubble sort)
2. Sort by Train name (Selection Sort)
3. Sort by Available Seats (Insertion Sort)
4. Sort by Total Seats (Merge Sort)
5. Sort by Start Station (Quick Sort)
0. Back

Choose sorting method: 4

Trains sorted using Merge Sort.

ID	Train	Route	Available/Total Seats
T007	Hill Country Express	Kandy -> Badulla	180/250
T006	Ella Odyssey	Colombo Fort -> Badulla	200/280
T004	Gampaha-Kandy Express	Gampaha -> Kandy	0/300
T009	Kandy-Colombo Intercity	Kandy -> Colombo Fort	220/300
T008	Badulla Night Mail	Badulla -> Colombo Fort	250/400
T002	Kandy Intercity	Colombo Fort -> Kandy	350/400
T005	Udarata Menike	Colombo Fort -> Badulla	300/420
T003	Main Line Express	Colombo Fort -> Polgahawela	400/450
T001	Podi Menike	Colombo Fort -> Badulla	450/500

===== SORT TRAINS =====

1. Sort by Train ID (Bubble sort)
2. Sort by Train name (Selection Sort)
3. Sort by Available Seats (Insertion Sort)
4. Sort by Total Seats (Merge Sort)
5. Sort by Start Station (Quick Sort)
0. Back

Choose sorting method: 5

Trains sorted using Quick Sort.

ID	Train	Route	Available/Total Seats
T008	Badulla Night Mail	Badulla -> Colombo Fort	250/400
T006	Ella Odyssey	Colombo Fort -> Badulla	200/280
T002	Kandy Intercity	Colombo Fort -> Kandy	350/400
T005	Udarata Menike	Colombo Fort -> Badulla	300/420
T003	Main Line Express	Colombo Fort -> Polgahawela	400/450
T001	Podi Menike	Colombo Fort -> Badulla	450/500
T004	Gampaha-Kandy Express	Gampaha -> Kandy	0/300
T007	Hill Country Express	Kandy -> Badulla	180/250
T009	Kandy-Colombo Intercity	Kandy -> Colombo Fort	220/300

Train Management

```
↑
↓
≡
≡
≡
≡
≡
===== ADMIN MENU =====
1. Manage trains
2. Manage stations and routes
3. Manage bookings
4. Users and activity
5. Sort trains
0. Log out
Choose an option: 1

--- TRAIN MANAGEMENT ---
1. View all trains
2. Search train by ID
3. Add train
4. Remove train
5. Demonstrate BST traversals
0. Back
Choose an option: 1

ID | Train | Route | Available/Total Seats
T001 | Podi Menike | Colombo Fort -> Badulla | 450/500
T002 | Kandy Intercity | Colombo Fort -> Kandy | 350/400
T003 | Main Line Express | Colombo Fort -> Polgahawela | 400/450
T004 | Gampaha-Kandy Express | Gampaha -> Kandy | 0/300
T005 | Udarata Menike | Colombo Fort -> Badulla | 300/420
T006 | Ella Odyssey | Colombo Fort -> Badulla | 200/280
T007 | Hill Country Express | Kandy -> Badulla | 180/250
T008 | Badulla Night Mail | Badulla -> Colombo Fort | 250/400
T009 | Kandy-Colombo Intercity | Kandy -> Colombo Fort | 220/300
```

BST Traversal

```
--- TRAIN MANAGEMENT ---
1. View all trains
2. Search train by ID
3. Add train
4. Remove train
5. Demonstrate BST traversals
0. Back
Choose an option: 5
1. In-order
2. Pre-order
3. Post-order
Traversal type: 1

--- IN-ORDER TRAVERSAL ---
Train ID: T001
Train Name: Podi Menike
Start Station: Colombo Fort
Destination: Badulla
Total Seats: 500
Available Seats: 450
Train ID: T002
Train Name: Kandy Intercity
Start Station: Colombo Fort
Destination: Kandy
Total Seats: 400
Available Seats: 350
Train ID: T003
Train Name: Main Line Express
Start Station: Colombo Fort
Destination: Polgahawela
Total Seats: 450
Available Seats: 400
```


Available Seats: 400
Train ID: T004
Train Name: Gampaha-Kandy Express
Start Station: Gampaha
Destination: Kandy
Total Seats: 300
Available Seats: 0
Train ID: T005
Train Name: Udarata Menike
Start Station: Colombo Fort
Destination: Badulla
Total Seats: 420
Available Seats: 300
Train ID: T006
Train Name: Ella Odyssey
Start Station: Colombo Fort
Destination: Badulla
Total Seats: 280
Available Seats: 200
Train ID: T007
Train Name: Hill Country Express
Start Station: Kandy
Destination: Badulla
Total Seats: 250
Available Seats: 180
Train ID: T008
Train Name: Badulla Night Mail
Start Station: Badulla
Destination: Colombo Fort
Total Seats: 400
Available Seats: 250
Train ID: T009
Train Name: Kandy-Colombo Intercity

Start Station: Kandy
Destination: Badulla
Total Seats: 250
Available Seats: 180
Train ID: T008
Train Name: Badulla Night Mail
Start Station: Badulla
Destination: Colombo Fort
Total Seats: 400
Available Seats: 250
Train ID: T009
Train Name: Kandy-Colombo Intercity
Start Station: Kandy
Destination: Colombo Fort
Total Seats: 300
Available Seats: 220

```
0. Back
Choose an option: 5
1. In-order
2. Pre-order
3. Post-order
Traversal type: 2

--- PRE-ORDER TRAVERSAL ---
Train ID: T001
Train Name: Podi Menike
Start Station: Colombo Fort
Destination: Badulla
Total Seats: 500
Available Seats: 450
Train ID: T002
Train Name: Kandy Intercity
Start Station: Colombo Fort
Destination: Kandy
Total Seats: 400
Available Seats: 350
Train ID: T003
Train Name: Main Line Express
Start Station: Colombo Fort
Destination: Polgahawela
Total Seats: 450
Available Seats: 400
Train ID: T004
Train Name: Gampaha-Kandy Express
Start Station: Gampaha
Destination: Kandy
Total Seats: 300
Available Seats: 0

Train Name: Udarata Menike
Start Station: Colombo Fort
Destination: Badulla
Total Seats: 420
Available Seats: 300
Train ID: T006
Train Name: Ella Odyssey
Start Station: Colombo Fort
Destination: Badulla
Total Seats: 280
Available Seats: 200
Train ID: T007
Train Name: Hill Country Express
Start Station: Kandy
Destination: Badulla
Total Seats: 250
Available Seats: 180
Train ID: T008
Train Name: Badulla Night Mail
Start Station: Badulla
Destination: Colombo Fort
Total Seats: 400
Available Seats: 250
Train ID: T009
Train Name: Kandy-Colombo Intercity
Start Station: Kandy
Destination: Colombo Fort
Total Seats: 300
Available Seats: 220
```

```
Choose an option: 5
1. In-order
2. Pre-order
3. Post-order
Traversal type: 3

--- POST-ORDER TRAVERSAL ---
Train ID: T009
Train Name: Kandy-Colombo Intercity
Start Station: Kandy
Destination: Colombo Fort
Total Seats: 300
Available Seats: 220
Train ID: T008
Train Name: Badulla Night Mail
Start Station: Badulla
Destination: Colombo Fort
Total Seats: 400
Available Seats: 250
Train ID: T007
Train Name: Hill Country Express
Start Station: Kandy
Destination: Badulla
Total Seats: 250
Available Seats: 180
Train ID: T006
Train Name: Ella Odyssey
Start Station: Colombo Fort
Destination: Badulla
Total Seats: 280
Available Seats: 200
Train ID: T005
```

```
Available Seats: 200
Train ID: T005
Train Name: Udarata Menike
Start Station: Colombo Fort
Destination: Badulla
Total Seats: 420
Available Seats: 300
Train ID: T004
Train Name: Gampaha-Kandy Express
Start Station: Gampaha
Destination: Kandy
Total Seats: 300
Available Seats: 0
Train ID: T003
Train Name: Main Line Express
Start Station: Colombo Fort
Destination: Polgahawela
Total Seats: 450
Available Seats: 400
Train ID: T002
Train Name: Kandy Intercity
Start Station: Colombo Fort
Destination: Kandy
Total Seats: 400
Available Seats: 350
Train ID: T001
Train Name: Podi Menike
Start Station: Colombo Fort
Destination: Badulla
Total Seats: 500
Available Seats: 450
```

Booking Management

===== AVAILABLE STATIONS =====

CMB - Colombo Fort
GPH - Gampaha
VGD - Veyangoda
PLG - Polgahawela
RBG - Rambukkana
KDY - Kandy
NWP - Nawalapitiya
HTN - Hatton
NUO - Nanu Oya
ELL - Ella
BDL - Badulla

Enter start station ID or name: *CMB*

Enter destination station ID or name: *KDY*

ID | Train | Route | Available/Total Seats

T006 | Ella Odyssey | Colombo Fort -> Badulla | 199/280

T002 | Kandy Intercity | Colombo Fort -> Kandy | 350/400

T005 | Udarata Menike | Colombo Fort -> Badulla | 300/420

T001 | Podi Menike | Colombo Fort -> Badulla | 449/500

You can use following train(s) from Colombo Fort to Kandy:

- T006 Ella Odyssey | Board at Colombo Fort | Leave at Kandy | Full service: Colombo Fort -> Badulla | Available seats: 199
- T002 Kandy Intercity | Board at Colombo Fort | Leave at Kandy | Full service: Colombo Fort -> Kandy | Available seats: 350
- T005 Udarata Menike | Board at Colombo Fort | Leave at Kandy | Full service: Colombo Fort -> Badulla | Available seats: 300
- T001 Podi Menike | Board at Colombo Fort | Leave at Kandy | Full service: Colombo Fort -> Badulla | Available seats: 449

Enter the train ID to book: *T006*

Enter seat numbers separated by commas (example: 1,2): *0*

Booking failed: Seat 0 is outside this train's capacity.

=====

TRAIN TICKET

=====

Booking ID : B0001
User ID : Tharu
Train ID : T006
From : Colombo Fort
To : Kandy
Seats : [3]
Distance : 90.0 km
Fare per km : Rs. 10.0
Total Price : Rs. 900.0

=====

BOOKING CONFIRMED

=====

Booking ID: B0001
Seats: [3]
Booking confirmed successfully.

AVL Tree

```
--- BOOKING MANAGEMENT ---
1. Search booking by ID
2. View all bookings
3. View waiting list
4. Demonstrate AVL traversals
5. View booked-seat set
0. Back
Choose an option: 4
1. In-order
2. Pre-order
3. Post-order
Traversal type: 1
```

```
--- IN-ORDER TRAVERSAL ---
B0001
B0002
```

```
--- BOOKING MANAGEMENT ---
1. Search booking by ID
2. View all bookings
3. View waiting list
4. Demonstrate AVL traversals
5. View booked-seat set
0. Back
Choose an option: 4
1. In-order
2. Pre-order
3. Post-order
Traversal type: 2
```

```
--- PRE-ORDER TRAVERSAL ---
B0001
B0002
```

```

--- BOOKING MANAGEMENT ---
1. Search booking by ID
2. View all bookings
3. View waiting list
4. Demonstrate AVL traversals
5. View booked-seat set
0. Back
Choose an option: 4
1. In-order
2. Pre-order
3. Post-order
Traversal type: 3

--- POST-ORDER TRAVERSAL ---
B0002
B0001

```

Validation

```

===== AVAILABLE STATIONS =====
CMB - Colombo Fort
GPH - Gampaha
VGD - Veyangoda
PLG - Polgahawela
RGG - Rambukkana
KDY - Kandy
NWP - Nawalapitiya
HTN - Hatton
NUO - Nanu Oya
ELL - Ella
BDL - Badulla
Enter start station ID or name: CMB
Enter destination station ID or name: KDY

ID | Train | Route | Available/Total Seats
T006 | Ella Odyssey | Colombo Fort -> Badulla | 199/280
T002 | Kandy Intercity | Colombo Fort -> Kandy | 350/400
T005 | Udarata Menike | Colombo Fort -> Badulla | 300/420
T001 | Podi Menike | Colombo Fort -> Badulla | 449/500

You can use following train(s) from Colombo Fort to Kandy:
- T006 Ella Odyssey | Board at Colombo Fort | Leave at Kandy | Full service: Colombo Fort -> Badulla | Available seats: 199
- T002 Kandy Intercity | Board at Colombo Fort | Leave at Kandy | Full service: Colombo Fort -> Kandy | Available seats: 350
- T005 Udarata Menike | Board at Colombo Fort | Leave at Kandy | Full service: Colombo Fort -> Badulla | Available seats: 300
- T001 Podi Menike | Board at Colombo Fort | Leave at Kandy | Full service: Colombo Fort -> Badulla | Available seats: 449
Enter the train ID to book: T006
Enter seat numbers separated by commas (example: 1,2): 0
Booking failed: Seat 0 is outside this train's capacity.

```

9. Challenges and Solutions

When we were building the Railway Reservation System, the team kind of ran into a bunch of issues—mostly with data handling, the algorithms themselves, reservations, and overall speed. So yeah, the tricky bits and what we tried to solve them with are laid out like this.

9.1 Managing Train Data

Challenge:

Keeping track of a changing number of trains, and then being able to add, remove, search, and show them without it turning into a mess.

Solutions:

We used a linked list to store train details in a more dynamic way. Operations like adding or deleting trains were put into the ***TrainLinkedList*** class. This makes the management way more flexible, no fixed-size limitation hanging over it.

9.2 Sorting Train Information

Challenge:

Sorting the trains according to different needs, but still keeping the code a bit organized and not tangled.

Solutions:

We created a dedicated sorting class just for sorting-related tasks. Merge Sort was implemented, and Bubble Sort too. Then ***sortTrains()*** methods were added to both ***UserMenu*** and ***AdminMenu***. Also, the sorting logic was kept separate from ***TrainLinkedList***, so ***TrainLinkedList*** stays focused on storage.

9.3 Finding Routes Between Stations

Challenge:

Figure out possible routes between connected stations efficiently, so the system doesn't crawl every time someone searches.

Solutions:

We modeled stations and connections as a graph. BFS was used for level-by-level exploration. DFS was used when we needed deeper route probing. Together these approaches help reveal feasible routes without wasting too much time.

9.4 Managing Seat Reservations

Challenge:

Avoiding duplicate bookings and still keeping seat availability correct, even when multiple people are trying.

Solutions:

Before confirming a reservation, we checked whether the seat is actually available. Seats that were already taken were not allowed to be chosen. After a booking, and also after cancellations, seat information was updated.

9.5 Managing Waiting Passengers

Challenge:

When all seats are full, handling passengers properly and making sure the process stays fair.

Solutions:

A queue was used for the waiting list. We followed FIFO, the First-In-First-Out idea. So once seats open up, passengers are processed based on the exact waiting order.

9.6 Efficient Booking Management

Challenge:

Handling and searching lots of booking records without slowdowns.

Solutions:

An AVL tree was chosen for booking management. We implemented insertion and searching in an efficient manner. Also, the tree balance was maintained to keep performance from dropping.

9.7 Managing User Information

Challenge:

Finding user information quickly as the number of users keeps growing.

Solutions:

We implemented a hash table for storing user records. User identifiers were used as the keys, so access is direct. Hashing gives faster retrieval compared with sequential search, especially when the user count gets larger.

10. Contribution Sheet

Member Number	Student ID	Student Name	Assigned Responsibilities (Completion is 100%)	Extra Contribution
1	CIT-25-01-0177	Rashith Randil	Arrays, Linked Lists, Linear Search, and complexity analysis	Implemented user /admin menu and system architecture
2	CIT-25-01-0648	Dinaya Ranasinghe	Hash Tables, Set ADT, Quick Sort complexity analysis	Implemented user menu And business logic
3	CIT-25-01-0755	Harindi Kaveesha	Graphs (Graph Representation, BFS, DFS), Merge Sort, complexity analysis	Implement distance/Pricing, Implemented admin menu
4	CIT-25-01-0084	Oshini Akarsha	Stack, Queue, Bubble Sort, and complexity analysis	Implemented user menu and business logic
5	CIT-25-01-0769	Sakunika Kavindi	Binary Search Tree (BST), Selection Sort complexity analysis	Implemented admin menu and business logic
6	CIT-25-01-0828	Tharushi Dewmini	AVL Tree, Insertion Sort complexity analysis	Implemented admin menu and business logic

11. Conclusion

The Railway Reservation System was developed as a project to apply the data structures and algorithms learned in **CCS2300 – Data Structures and Algorithms** into a practical system.

Different data structures were implemented in different parts of the project, such as for station records, train records, user details, bookings, waiting lists, seat reservations, tree operations, and route finding algorithms. The project was tested, and the main functions, such as registration and login, train and station functions, booking and waiting lists, and route finding, all performed according to expectations.

Through the development of this project, we were able to gain an understanding of how different data structures and algorithms can be combined to solve a real-world problem. Additionally, we gained experience in selecting the different data structures for different parts of the project.

In the future, the system can be improved by using a relational database like MySQL or PostgreSQL, adding password hashing and authentication systems, creating a graphical user interface, implementing role-based access control, and adding an automatic data backup system.

Overall, the project was successful in demonstrating how data structures and algorithms can be used in combination to create a Railway Reservation System.

12. Reference List

1. Oracle. **Java Documentation.**

2. **GitHub. Railway Reservation System** – Group Project Repository.

[Railway Reservation System GitHub Repository] (https://github.com/RashithRandle/railway-reservation-system.git?utm_source=chatgpt.com)

3. **OpenAI. ChatGPT.** Used as an AI-assisted tool for understanding data structures, code explanations, debugging support, and documentation preparation.

[ChatGPT](https://chatgpt.com/?utm_source=chatgpt.com)